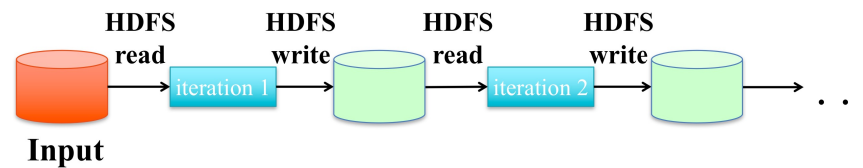




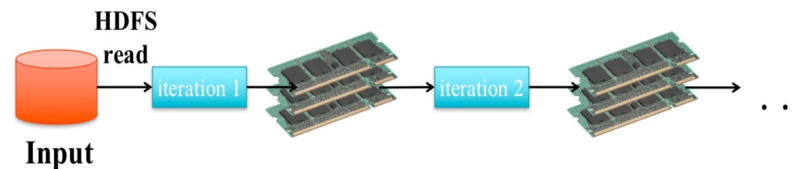


Introduction à Spark

- Le moteur d'exécution MapReduce est de moins en moins utilisé du fait de sa lenteur. En effet, les résultats des phases intermédiaires sont écrites et relues à partir de HDFS. Des accès aux données qui coûtent et pénalisent les temps de traitement.



- En 2009, une équipe de recherche de Berkley va travailler sur le développement d'un gestionnaire de ressources appelé Mesos. Pour la validation de Mesos, ils vont implémenter un moteur d'exécution sous le nom de Spark. Celui-ci contrairement au MapReduce de HADOOP va conserver les résultats intermédiaires en mémoire ce qui en fera quelques années plus tard la référence en terme de moteur d'exécution.



- De nombreux projets de l'éco-système HADOOP vont abandonner le moteur MapReduce de HADOOP au profit de Spark comme Hive, Mahout et bien d'autres.
- Spark est un système distribué d'analyse de données « in-memory » et extensible.
- Il supporte plusieurs langages de programmation Java, Scala, Python.

Introduction à Spark

- Spark s'inspire fortement de HADOOP
 - L'utilisation de matériel commun pour le traitement **mais ne gère pas le stockage des données**
 - Les pannes matérielles sont gérées au niveau logiciel
 - Exécuter le code applicatif au plus prêt des données
 - Séparer la logique du traitement des données de la logique de distribution du calcul
 - Intègre son propre cluster manager 'Spark standalone' lequel est interchangeable avec YARN / MESOS
 - Cluster mode
 - Local mode
- Les améliorations apportées par Spark
 - Développé en SCALA, langage basé sur une JVM.
 - 100 lignes de code JAVA peut être traduit en une dizaine de lignes en SCALA
 - Hérite des avantages de la programmation fonctionnelle
 - Le calcul est évalué comme une fonction mathématique
 - Le développement d'applications parallélisées est plus facile
 - Concept de Resilient Distributed Dataset
- Spark est adapté pour les traitements batch (par lot) mais ne convient pour les systèmes en temps réel. On peut classer les applications en temps réel comme des applications qui traitent les données en flux tendu avec un temps de traitement inférieur à la dizaine de milli-seconde. Vous pouvez aussi entendre de traitements pseudo temps-réel ou le temps de traitement est compris entre 10 ms et 50 ms. Ce type de traitement peut être géré par Spark avec ses fonctionnalités de streaming.

Introduction à Spark

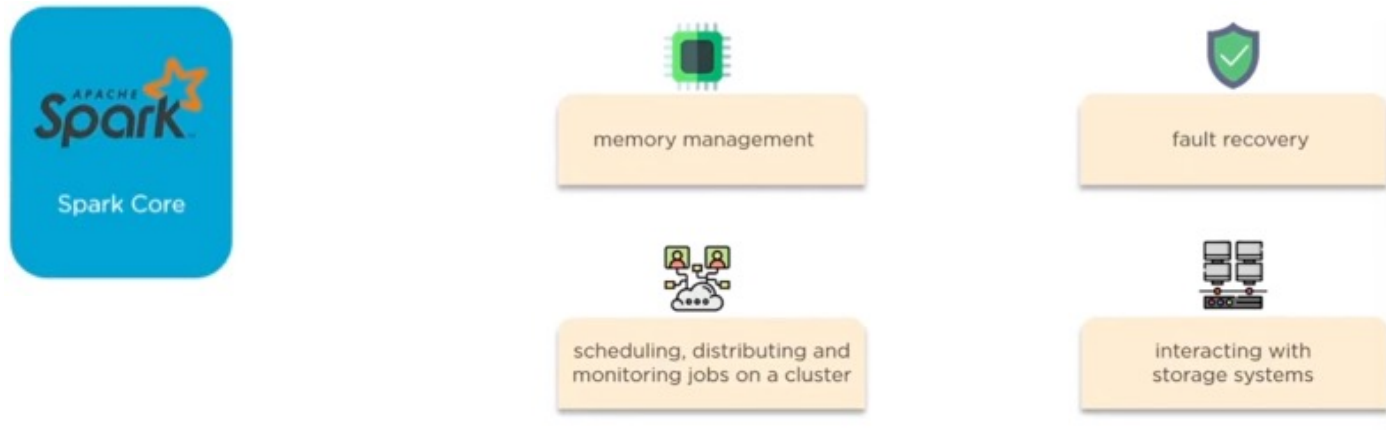
- L'idée de Spark est d'appliquer les mêmes opérations à tous les éléments d'un jeu de données.
- Bien entendu, au même titre que MapReduce, Spark gère la tolérance aux pannes et peut resoumettre une tâche qui échoue.
- Spark couvre une large variété de traitement :
 - Batch programs
 - Interactive SQL
 - Streaming
 - Graph processing
- Spark se compose de 5 éléments :



Spark Eco-Système



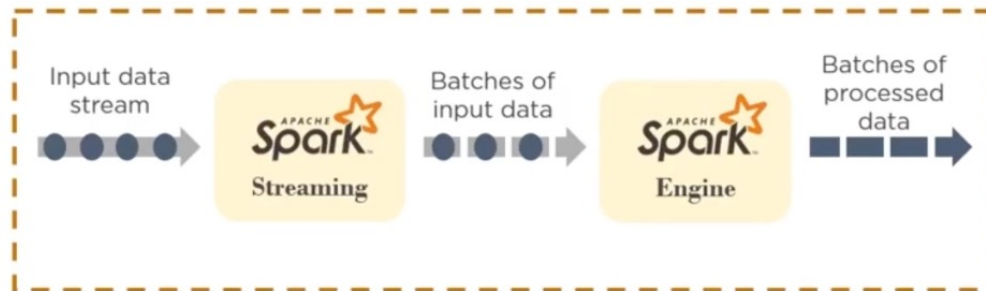
Spark Core



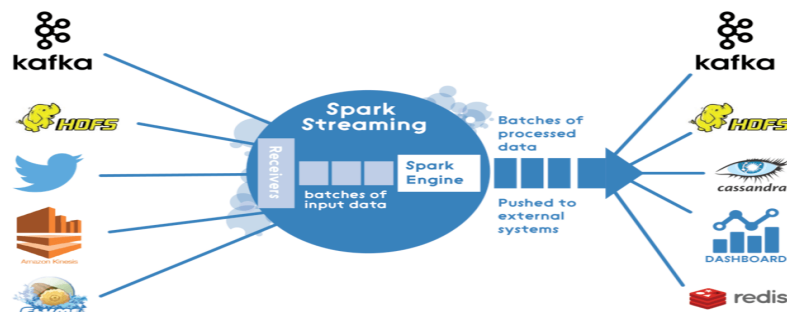
Spark Core gère plusieurs aspects fondamentaux du fonctionnement de Spark :

- Spark optimise l'utilisation de la **mémoire RAM** pour accélérer les calculs.
- Si un nœud tombe en panne, Spark **reconstruit les données** à partir des RDDs.
- Il parallélise l'exécution d'un job/programme Spark.
- Il gère l'interaction avec les systèmes de stockage.

Spark Streaming



- Les traitements par flux permettent de traiter les données par un intervalle régulier. Par exemple, Spark va collecter les données provenant de sources de données comme Kafka , une base NoSQL, des réseaux sociaux toutes les 5s et appliquer un traitement qui peut être redirigé dans une source externe comme HDFS, une base de données NoSQL ...
- Le traitement est continu. En cas de panne ou d'arrêt, il est possible de configurer Spark pour mettre en place des points de contrôle.



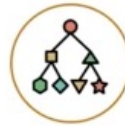
Spark Machine Learning



It contains machine learning libraries that have an implementation of various machine learning algorithms



Clustering

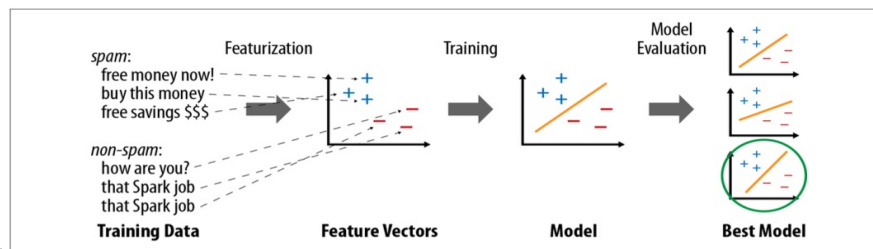


Classification

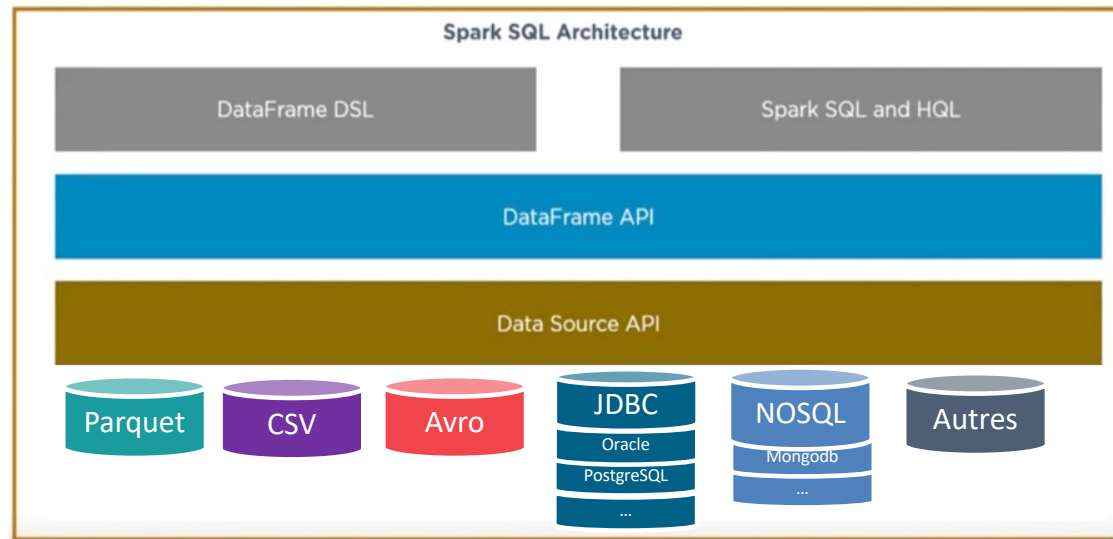
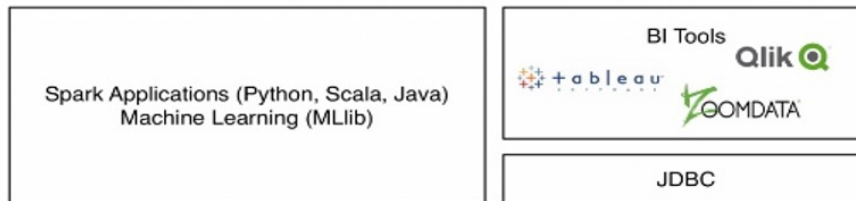


Collaborative Filtering

- Le module Spark MLlib offre des fonctionnalités d'apprentissage dans plusieurs domaines. La documentation disponible sur le site Web de Spark présente les types de données utilisés (par exemple, les vecteurs et la structure LabeledPoint). Ce module offre des fonctionnalités qui incluent:
 - Statistics
 - Classification
 - Regression
 - Collaborative Filtering
 - Clustering
 - Dimensionality Reduction
- Notez qu'une librairie spécifique au machine learning a été développée sous le nom d'Apache Mahout laquelle est compatible avec Spark.



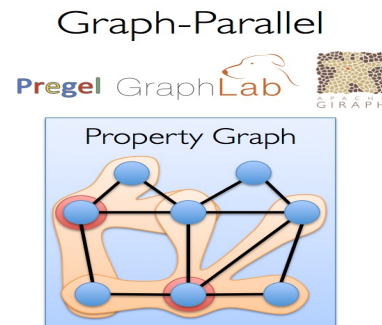
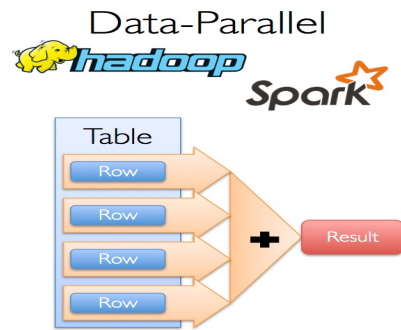
Spark SQL



Spark SQL est utilisé pour les traitements de données structurés et semi-structurés.

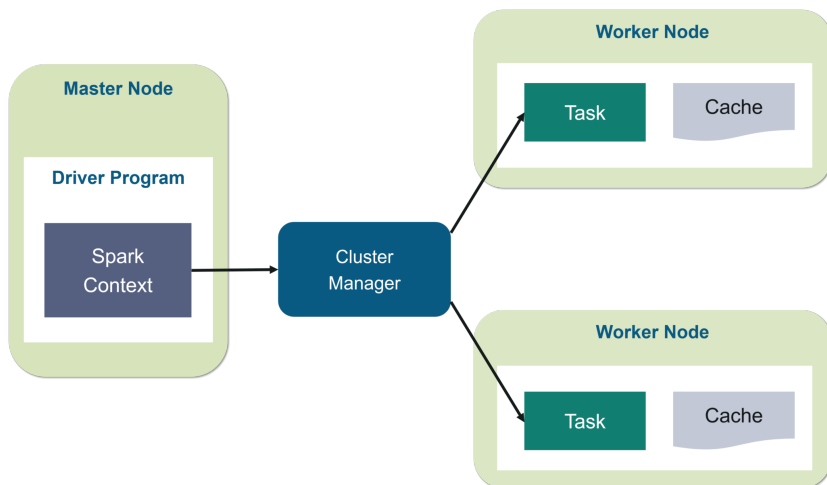
- Depuis la version 1.3, Spark intègre la notion de dataframe afin que les données puissent être traitées sous forme tabulaire et que des fonctions tabulaires (telles que select, filter, groupBy) puissent être utilisées pour traiter les données.
- Le module Spark SQL est compatible avec une large variété de formats Parquet, Avro, ORC, JSON, CSV ...
- Spark SQL est une interface qui permet d'exécuter du SQL quelque soit le format du fichier.
- La diversité des formats permet à Spark d'être flexible mais surtout de faciliter l'exploitation des données.
- Spark SQL permet aussi de s'interfacer avec Hive et des bases de données

Spark GraphX



- Spark int gre un module GraphX qui vous permet de traiter vos donn es sous forme de graphes en repr sentant celles-ci sous forme d'arcs et de sommets comme avec Neo4J.
- GraphX est d crit comme une solution permettant de traiter de gros volume de donn es mais cela reste   confirmer en pratique. En effet, les donn es ne sont pas structur es comme dans Neo4J sous forme de graphe ce qui rend les performances moindres.

Spark architecture



Le Master Node contient le **Driver Program** qui contrôle l'exécution du job SPARK.

La connexion entre le programme et le cluster est réalisée par le **SparkContext**. Le **SparkContext** est le point d'entrée principal pour interagir avec Spark.

Le **Driver** soumet les jobs au **Cluster Manager** via le **SparkContext**.

Le Cluster Manager est responsable de l'**allocation des ressources** aux worker nodes.

Le Cluster Manager peut être **YARN, Mesos, Kubernetes** ou le **Cluster Manager intégré de Spark** (Spark standalone pour le développement uniquement).

Le Cluster Manager **distribue les tâches (tasks)** aux worker nodes disponibles.

Chaque **Worker Node** exécute un **Executor**, qui :

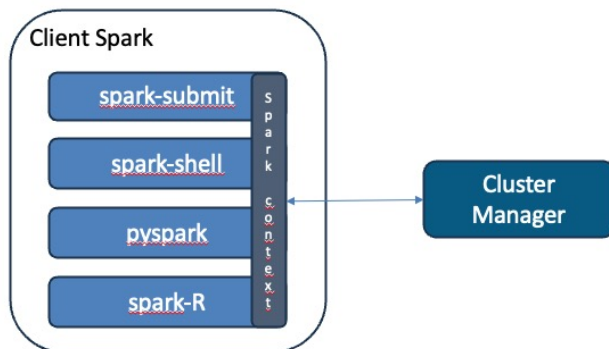
1. **Reçoit des tâches** du Cluster Manager.
2. **Stocke les données mises en cache** (si cache() est utilisé).
3. **Exécute des tâches en parallèle**.

Un worker peut exécuter plusieurs tasks en parallèle grâce à l'utilisation de partitions de données.

Spark architecture

- Apache Spark propose plusieurs **clients** pour interagir avec le cluster et exécuter des jobs. Ces clients permettent d'utiliser Spark en **mode interactif, batch ou via des APIs externes**.

Client	Description	Langages	Cas d'utilisation
Spark Shell	Shell interactif pour Spark en Scala	Scala	Exploration de données, tests rapides
PySpark	Shell interactif pour Spark en Python	Python	Data Science, Machine Learning
Spark Submit	Soumission de jobs en batch	Scala, Java, Python	Exécution de jobs en production
Livy	API REST pour soumettre des jobs Spark	Scala, Python	Intégration avec applications web
Zeppelin	Notebook interactif avec support natif de Spark	Scala, Python, SQL	Data Science, visualisation
Jupyter Notebook	Notebook interactif pour PySpark	Python	Machine Learning, analyse exploratoire



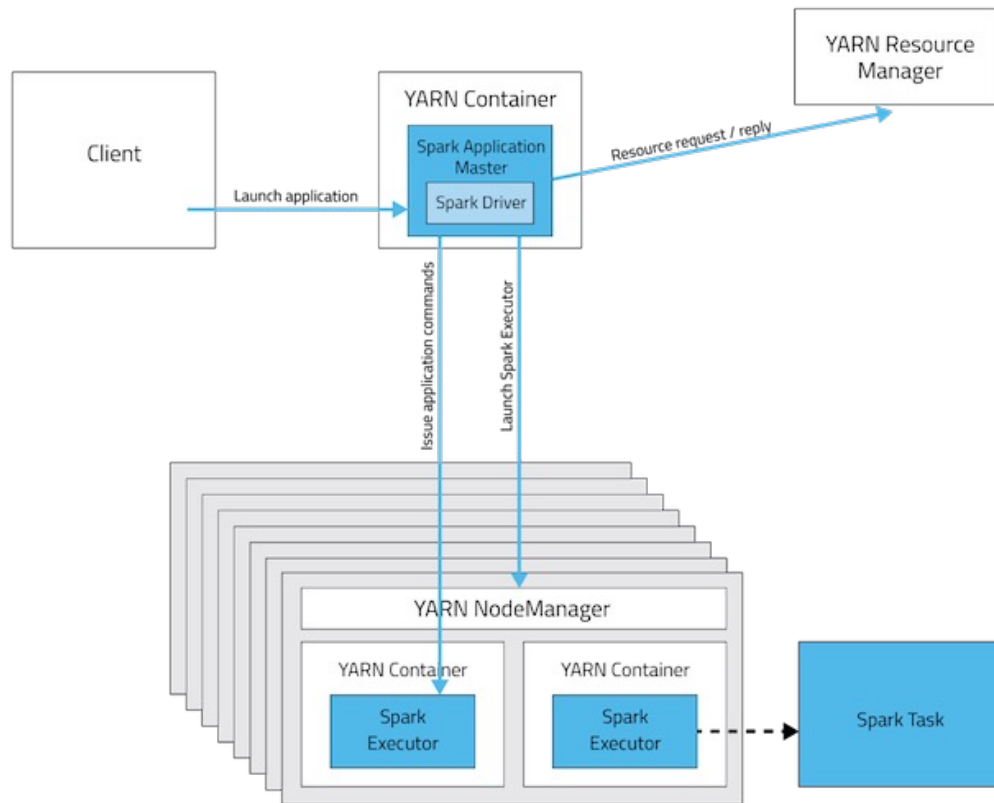
- Le client précise l'URL de connexion au cluster manager et les ressources dont il a besoin.

```
from pyspark import SparkContext, SparkConf

conf = SparkConf() \
    .setAppName("MonApplicationSpark") \
    .setMaster("master:9000") \ # Indique l'emplacement du cluster manager
    .set("spark.executor.memory", "1g") \ # Limite la mémoire des exécutants à 1 Go
    .set("spark.driver.memory", "512m") \ # Limite la mémoire du driver à 512 Mo

sc = SparkContext(conf=conf)
```


Spark architecture : Mode de Déploiement Cluster

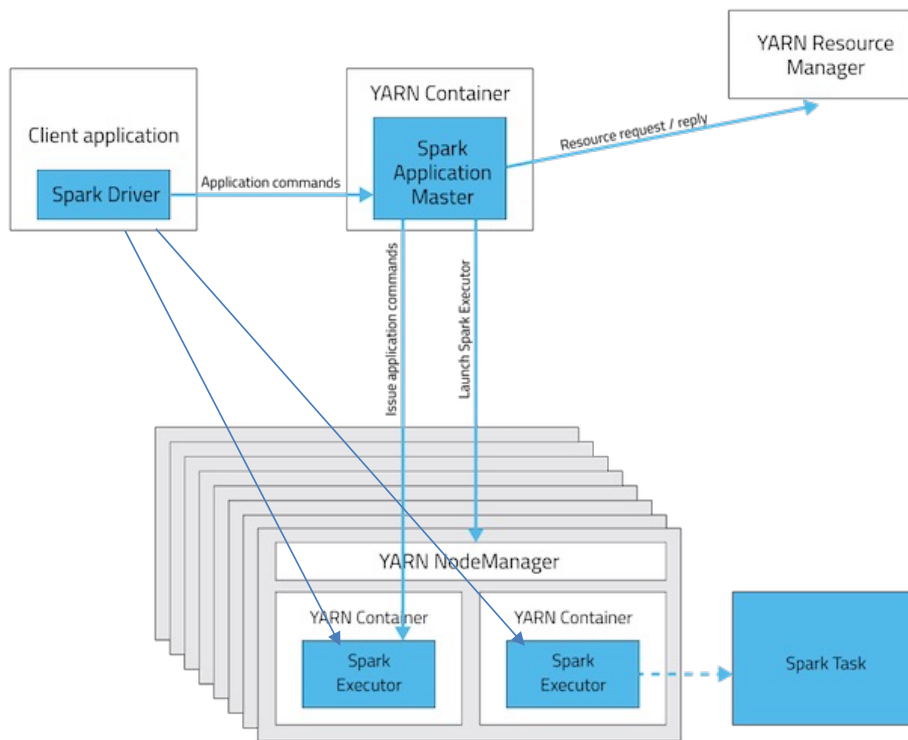


En **mode cluster**, le **driver Spark** s'exécute dans l'**ApplicationMaster** sur l'un des nœuds du cluster.

Un **unique processus dans un conteneur YARN** est responsable à la fois de **piloter l'application** et de **demander des ressources** à YARN.

Le **client qui soumet l'application n'a pas besoin de rester actif** pendant toute la durée d'exécution de l'application.

Spark architecture : Mode de Déploiement Client



En **mode client**, le **driver Spark** s'exécute sur la machine depuis laquelle le job est soumis.

L'**ApplicationMaster** a uniquement pour rôle de **demande des conteneurs d'exécuteurs** à YARN.

Une fois les conteneurs démarrés, le **client communique directement avec eux** pour planifier l'exécution des tâches.

Resilient Distributed Datasets (RDD)

- Les **RDDs (Resilient Distributed Datasets)** sont la structure de données fondamentale de Spark. Ils supportent **deux types d'opérations** :
 - 1. **Transformations** → Créent un **nouvel RDD** à partir d'un RDD existant.
 - 2. **Actions** → Déclenchent le **calcul** et renvoient un **résultat au driver**.



Resilient Distributed Datasets (RDD)



- Les **transformations** sont des opérations **paresseuses (lazy)** qui définissent un nouveau RDD **sans l'exécuter immédiatement**.
- Elles ne sont évaluées que lorsqu'une **action** est déclenchée.
 - **Exécution paresseuse** → Spark **n'exécute pas** les transformations immédiatement.
 - **RDDs immuables** → Chaque transformation crée **un nouvel RDD**, l'original restant inchangé.
 - **Optimisation DAG** → Spark optimise l'exécution avant d'exécuter le job.

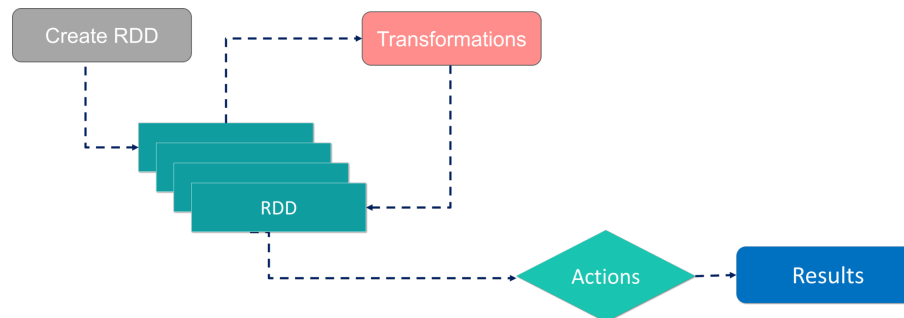
Transformation	Description	Exemple
<code>map()</code>	Applique une fonction à chaque élément	<code>rdd2= rdd1.map(x => x * 2)</code>
<code>filter()</code>	Filtre les éléments selon une condition	<code>rdd3= rdd2.filter(x => x > 10)</code>
<code>distinct()</code>	Supprime les doublons	<code>rdd4 = rdd3.distinct()</code>

Resilient Distributed Datasets (RDD)



- Les **actions** ordonnent à Spark d'exécuter les transformations **et de renvoyer les résultats au driver**.
 - **Déclenchent l'exécution du DAG (Directed Acyclic Graph).**
 - **Renvoient des résultats ou écrivent en sortie.**
 - **Effectuent des calculs distribués** sur les nœuds du cluster.

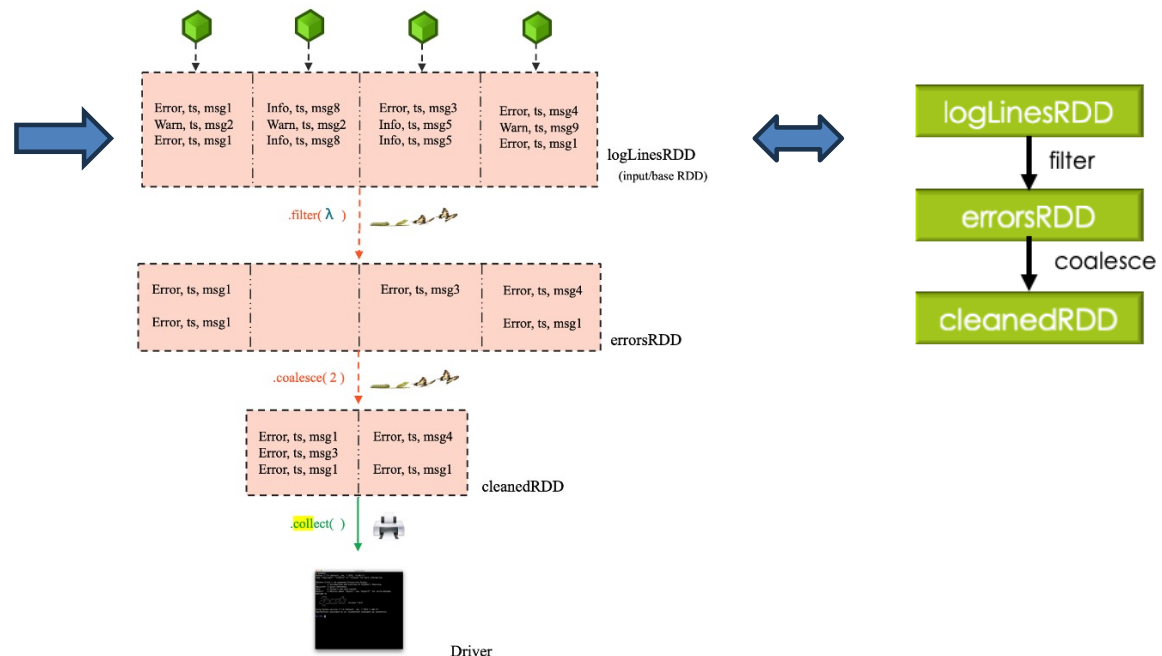
Action	Description	Exemple
<code>collect()</code>	Retourne tous les éléments au driver	<code>rdd.collect()</code>
<code>count()</code>	Retourne le nombre d'éléments	<code>rdd.count()</code>
<code>first()</code>	Retourne le premier élément	<code>rdd.first()</code>
<code>take(n)</code>	Retourne les n premiers éléments	<code>rdd.take(3)</code>
<code>reduce()</code>	Agrège les éléments avec une fonction	<code>rdd.reduce((a, b) => a + b)</code>



Resilient Distributed Datasets (RDD)

- Lorsque plusieurs transformations sont exécutées, Spark conserve les dépendances entre les RDD et exprime celles-ci sous forme de graphe (Directed Acyclic Graph DAG).
- SPARK utilise le graphe des dépendances pour optimiser le plan d'exécution mais aussi pour reconstruire les RDD en cas de panne d'une machine /worker.

```
logLinesRDD = sc.textFile("/path/to/logfile.txt",4)
errorsRDD = logLinesRDD.filter(lambda x: "error" in x)
cleanRDD = errorsRDD.coalesce( 2 )
cleanRDD.collect()
```



Resilient Distributed Dataset : RDD

Voici, un exemple d'un traitement Map et Reduce avec Spark.

```
from pyspark import SparkContext, SparkConf
conf = SparkConf() \
    .setAppName("MonApplicationSpark") \
    .setMaster("local[*]")
sc = SparkContext(conf=conf)

dataSrc = sc.textFile("mydata.txt")
lineLengths = dataSrc.map(lambda s: len(s))
totalLength = lineLengths.reduce(lambda a, b: a + b)
```

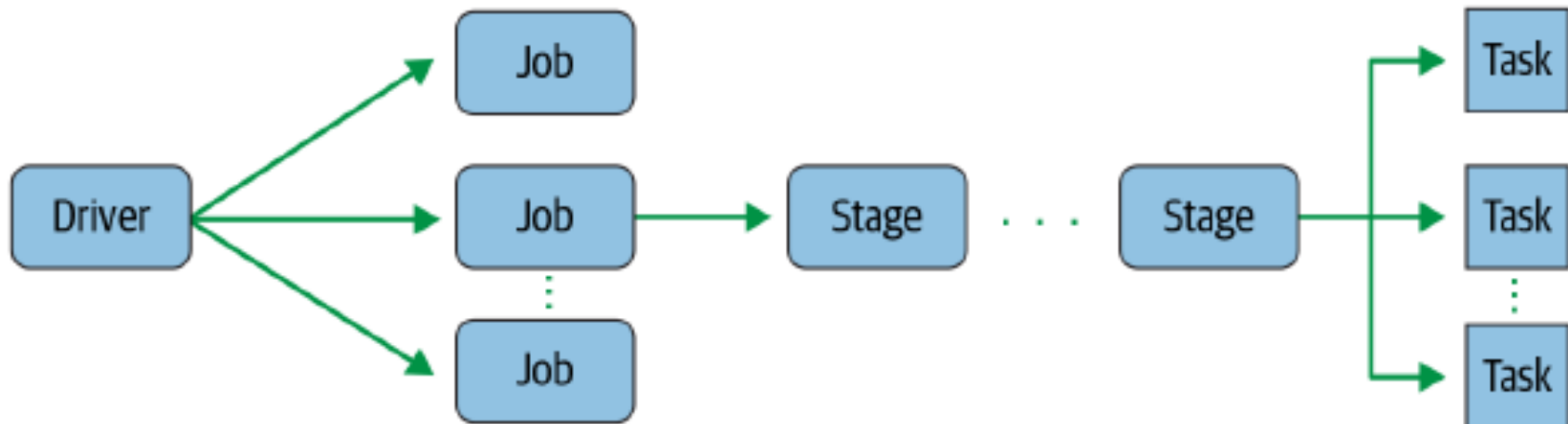
Que réalise ce programme ?

Le programme suivant compte le nombre de caractères dans un fichier.

Bien entendu, ce programme est décorrélé de la taille des données à traiter.

Ce programme fonctionnera aussi bien sur un cluster de 3 machines que sur N machines.

Qu'est ce qu'un programme SPARK ?



Qu'est ce qu'un programme SPARK ?

Un programme SPARK

Il peut opérer plusieurs RDD mais pour SPARK chaque RDD sera traité individuellement via un job SPARK.

Un job SPARK

Au sens SPARK, un job est le traitement d'un RDD lequel regroupe un ensemble de transformations afin d'obtenir un résultat.

Un job est créé lorsqu'une action est déclenchée (show(), collect(), write(), etc.).

Un job regroupe plusieurs stages.

Une étape / stage

Un stage est un ensemble de tâches qui peuvent être exécutées en parallèle.

Une tâche / task

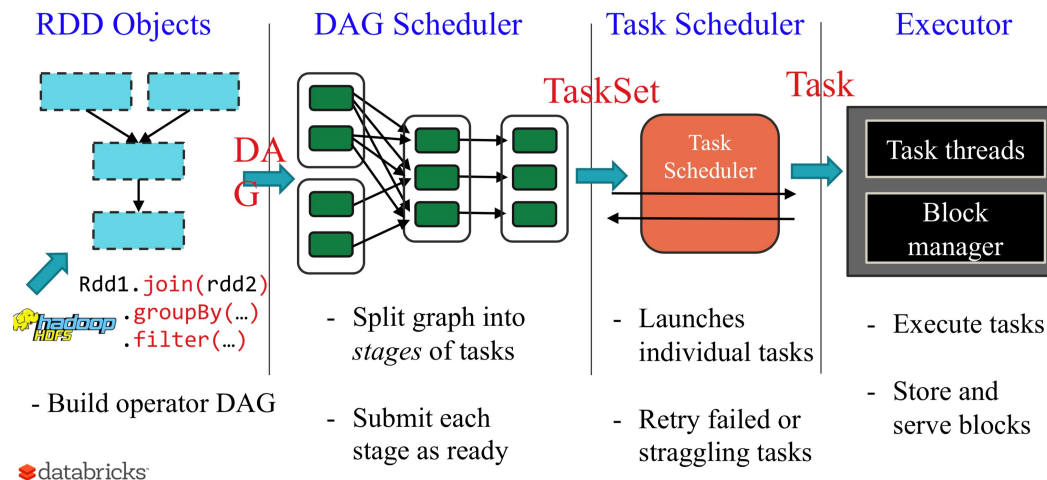
Une task est l'unité de travail la plus petite en Spark.

Chaque étape est divisée en plusieurs tasks, qui sont envoyées aux Executors.

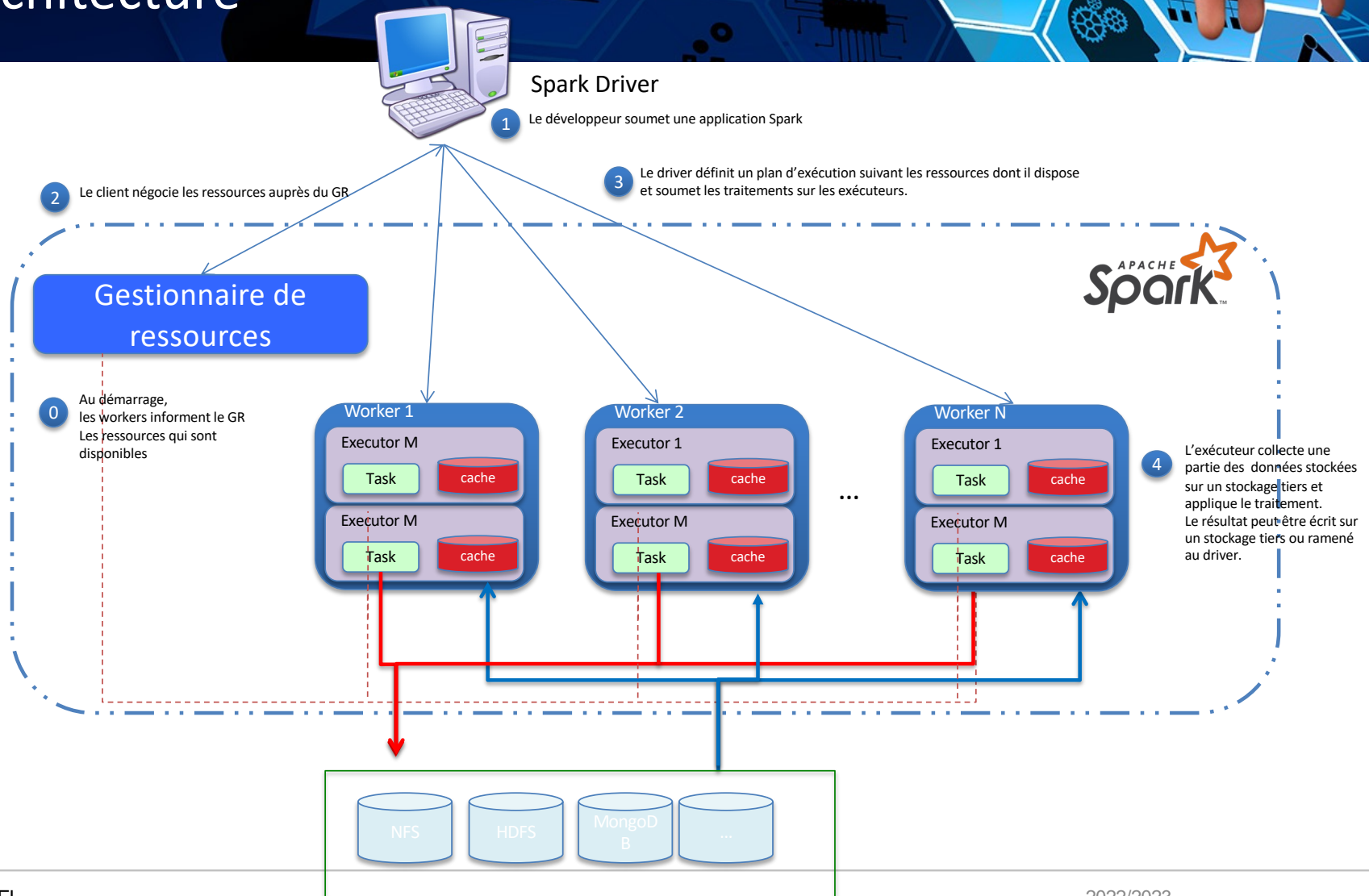
Qu'est ce qu'un programme SPARK ? Distributed Dataset RDD

- Voici comment Spark va interpréter votre programme et décharger le développeur de tous les aspects liés à la distribution du calcul.
 - Le plan d'exécution est appelé le Directed Acyclic Graph.
 - DAG est un graphe ou les sommets représentent le RDD et les arcs les opérations qui doivent être appliquées.
 - C'est à partir du DAG que l'ensemble des tâches seront réparties sur les exécuteurs.

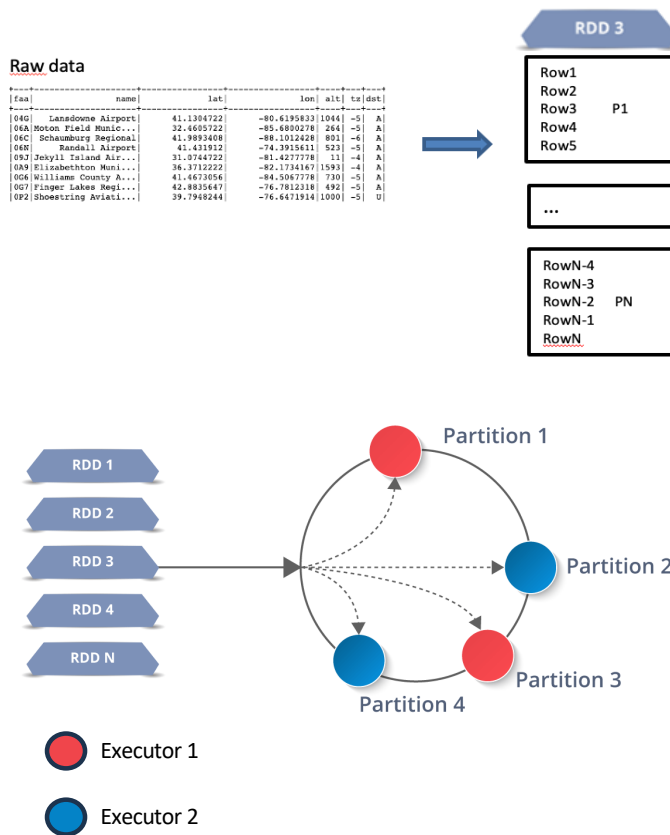
Scheduling Process



Spark architecture

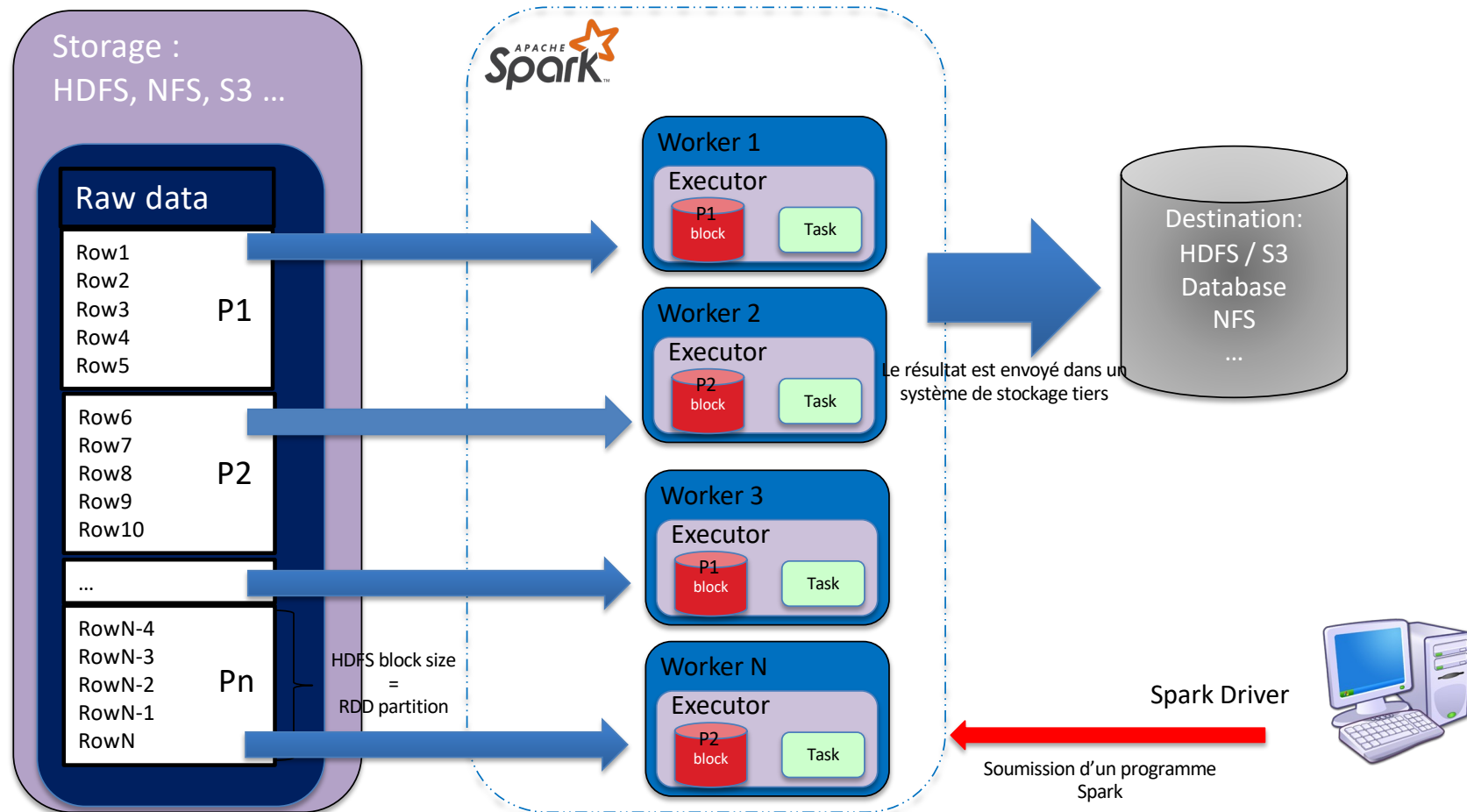


Resilient Distributed Dataset RDD



- Spark est conçu autour de la notion de **Resilient Distributed Dataset RDD** (jeux de données résilient et distribués).
- Le RDD représente vos données comme une collection qui peut être opérée en parallèle.
- Un RDD est une structure de données **immuable**. Une fois créé, il ne peut pas être modifié. Une opération qui modifie un RDD retourne un nouvel RDD.
- Spark s'appuie sur la notion de RDD pour distribuer les traitements. Le RDD est décomposé en partition ou chacune d'entre elle sera traitée indépendamment et en parallèle sur les executors/workers.
- Notez qu'il est possible d'avoir une correspondance entre les partitions du RDD et les blocs HDFS. Par exemple, HDFS stock les données en bloc de 128Mo et Spark va s'appuyer de ses échanges avec le Namenode pour établir une correspondance 1-1 entre les partitions du RDD et la tailles des blocs HDFS.

Resilient Distributed Dataset RDD



Les ressources avec Spark

- Un job SPARK a besoin de ressources CPU/RAM pour traiter les données.
- Avec Yarn, les ressources peuvent être imposées par l'administrateur en utilisant un planificateurs de ressources (FIFO, Capacity, Fair).
- Parmi les ressources accordées à un utilisateur, celui-ci peut personnaliser les ressources dont il a besoin.

```
conf = SparkConf() \  
    .setAppName("MonApplicationSpark") \  
    .setMaster("spark://smaster:7077") \  
    .set("spark.executor.memory", "1g") \  
    .set("spark.driver.memory", "512m") \  
sc = SparkContext(conf=conf)
```



Les ressources avec Spark

Vous pouvez soumettre de différents jobs :

- Tiny jobs, des jobs qui nécessitent peu de ressources, peu de tâches et peu de calcul.
- Fat jobs, des jobs qui vont traiter des gros volumes de données avec un grand nombre de tâches parallèles CPU intensif et donc demander beaucoup de ressources.

Les ressources demandées vont dépendre du type de jobs et des ressources disponibles.

Un exécuteur peut exécuter une ou plusieurs **tâches (tasks)** en parallèle en fonction du nombre de **cœurs (cores)** qui lui sont attribués.

Lorsqu'on soumet un job Spark, on peut configurer :

- **Le nombre d'exécuteurs** (--num-executors) limité au nombre de cœurs par machine.
- **Le nombre de cœurs par exécuteur** (--executor-cores)
- **La mémoire disponible par exécuteur** (--executor-memory)

Les Data Frames

- En 2015, Spark va s'inspirer de la notion de Data Frame en R et Python (Panda) pour fournir une nouvelle API plus simple à utiliser que les RDD.
- Les Data Frames offrent un haut niveau d'abstraction qui facilitent la programmation et améliorent les performances d'exécution quelque soit le langage de programmation.
 - L'utilisation des RDD en JAVA, Python ou Scala oblige Spark à convertir le code en java byte code. Dans le cas où vous programmez en python, Spark va utiliser la librairie Py4J pour convertir votre code en JAVA et ensuite générer le byte code. Ces phases de conversions ne sont pas optimales et se traduisent par des pertes de performances.
- On écrit de moins en moins de code.



```
private IntWritable one = new IntWritable(1);
private IntWritable output = new IntWritable();
protected void map(LongWritable key,
    Text value,
    Context context) {
    String[] fields = value.split("\t");
    output.set(Integer.parseInt(fields[1]));
    context.write(one, output);
}

IntWritable one = new IntWritable(1)
DoubleWritable average = new DoubleWritable();

protected void reduce(IntWritable key,
    Iterable<IntWritable> values,
    Context context) {
    int sum = 0;
    int count = 0;
    for (IntWritable value: values) {
        sum += value.get();
        count++;
    }
    average.set(sum / (double) count);
    context.write(key, average);
}
```



Using RDD

```
rdd = sc.textFile(...).map(_.split(" "))
rdd.map { x => (x(0), (x(1).toFloat, 1)) }.
  reduceByKey { case ((num1, count1), (num2, count2)) =>
    (num1 + num2, count1 + count2)
  }.
  map { case (key, (num, count)) => (key, num / count) }.
  collect()
```

```
rdd = sc.textFile(...).map(lambda s: s.split())
rdd.map(lambda x: (x[0], (float(x[1]), 1))).\
  reduceByKey(lambda t1, t2: (t1[0] + t2[0], t1[1] + t2[1])).\
  map(lambda t: (t[0], t[1][0] / t[1][1])).\
  collect()
```

Using DataFrames

```
import org.apache.spark.sql.functions._

val df = rdd.map(a => (a(0), a(1))).toDF("key", "value")
df.groupBy("key")
  .agg(avg("value"))
  .collect()
```

- Comme les RDD, les DataFrames sont paresseux. Les transformations contribuent au plan de requête, mais elles n'exécutent rien.
- Les actions provoquent l'exécution de la requête.

Transformation	Action
filter	Count
select	Collect
drop	Show
intersect	Head
join	take
...	..

Les Data Frames : exemple d'utilisation

- Création d'un DataFrame qui contient seulement les utilisateurs de moins de 18 ans.
 - `young = users.filter(users["age"] < 21)`
- Une syntaxe alternative similaire à la syntaxe Pandas en python
 - `young = users[users.age < 21]`
- Ajouter un an à tous les utilisateurs :
 - `young.select(young["name"], young["age"] + 1)`
- Compter le nombre d'utilisateur par genre
 - `young.groupBy("gender").count()`
- Joindre les utilisateurs de moins de 18 ans à un autre dataframe logs via le champ `userId`:
 - `young.join(log, logs["userId"] == users["userId"], "left_outer")`

- Création d'un DataFrame avec un objet SQLContext
 - Dans le shell Spark Scala ou pySpark, vous avez un SQLContext disponible automatiquement, sous la forme sqlContext.
 - Dans une application, vous pouvez facilement en créer un vous-même à partir d'un SparkContext.
 - L'API de source de données DataFrame est cohérent, quel que soit le format de données. Ouvrir une source de données fonctionne à peu près de la même manière, quoi qu'il arrive.

Les 4 lignes suivantes indiquent la configuration du cluster Spark lesquels ne sont pas nécessaires avec le shell pySpark.

```
from pySpark import SparkContext, SparkConf
conf = SparkConf().setAppName(appName).setMaster(master)
sc = SparkContext(conf=conf)
sqlContext = SQLContext(sc)
```

Création d'un DF à partir de données JSON et un autre à partir de données parquet.

```
df = sqlContext.read.parquet("/path/to/data.parquet")
df2 = sqlContext.read.json("/path/to/data.json")
```

- Les Data Frames ont une structure de table qui est décrite par un schéma.
- Spark peut inférer un schema à partir d'un fichier JSON ou Parquet.
- Pour voir le schema d'un DataFrame utilisez la méthode `printSchema()`

```
df.printSchema() root
|-- firstName: string (nullable = true)
|-- lastName: string (nullable = true)
|-- gender: string (nullable = true)
|-- age: integer (nullable = false)
```

- Bien entendu, il est possible de spécifier le schéma manuellement

TP : Spark DF

